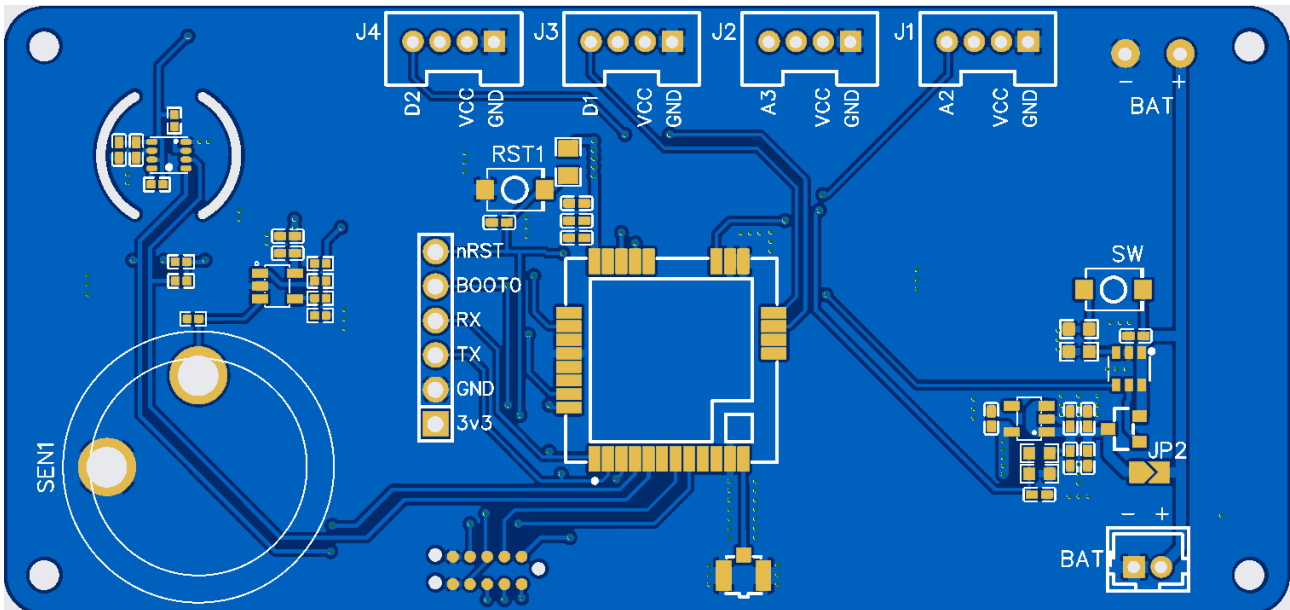


RAK3172 (STM32WLE5) – Complete Bring-up, UART Boot/Upload & Testgids (uitgebreid)

Versie: 2026-01-03 • Doel: naslagwerk om jouw custom RAK3172-PCB later direct weer te kunnen programmeren, debuggen en testen.

Deze gids vat samen wat we hebben uitgezocht: UART-aansluiting (incl. TRU COMPONENTS TC-9260976), bootloader-procedure, Arduino IDE setup, waarom de LoRaWAN-library eerst faalde, welke patches nodig waren, en de definitieve testsketch met stap-voor-stap uitleg.

0. Referentie: PCB-top view



Op de PCB zie je links de sensoraansluiting ('SEN1'), midden de RAK3172 footprint, en links-midden de programmeer/debug header met o.a. nRST, BOOT0, RX/TX, GND en 3V3.

1. Wat is het doel van deze testopzet?

Je wil het board zonder ST-Link kunnen programmeren en toch betrouwbaar debuggen. Daarom gebruiken we:

- STM32 System Bootloader via UART (BOOT0 + nRST) om firmware te uploaden.
- UART2 (PA2/PA3) voor debug output (via USB $\blacksquare$ TTL converter).
- Een vaste 8 $\blacksquare$ byte binaire payload (met versie/msgType byte) zodat Node $\blacksquare$ RED eenvoudig kan routeren/decoderen.

2. UART $\blacksquare$ aansluiting – TRU COMPONENTS TC $\blacksquare$ 9260976 (belangrijkste aandachtspunten)

Jouw converter heeft meerdere TX $\blacksquare$ pinnen. Dat is expres: één is 3,3V TTL en één is 5V TTL. Voor de RAK3172 (STM32) moet je altijd 3,3V logic gebruiken.

2.1 Pin-keuze op de TC $\blacksquare$ 9260976

Pin	Naam	Gebruik voor RAK3172	Opmerking
X2	TXD 3.3V	JA	Gebruik deze TX naar de RAK RX (PA3)

X3	RXD	JA	Gebruik deze RX vanaf de RAK TX (PA2)
X5	GND	JA	Altijd aansluiten (gemeenschappelijke massa)
X1	TTL driver input	JA (naar GND)	Aanbevolen: vastleggen op GND als je 5V driver niet gebruikt
X4	TXD 5V TTL	NEE	NIET aansluiten op STM32/RAK (5V logic)
X6	VBUS 5V	NEE	USB 5V output; niet nodig en niet veilig voor logic

2.2 Waarom X1 naar GND? (wat betekent 'TTL driver not used'?)

X1 is een ingang van een interne driver die bedoeld is om een 5V $\blacksquare$ TTL TX signaal te maken. Jij gebruikt die 5V $\blacksquare$ TX niet; je gebruikt de normale 3,3V TXD (X2). Als X1 los hangt kan hij ruis oppikken, waardoor de driver onbedoeld gedrag kan geven. Daarom leg je X1 vast op GND.

2.3 Correct kruisen van TX/RX (altijd)

Voor UART moet TX en RX altijd gekruist worden:

TC $\blacksquare$ 9260976	→	RAK3172 header
X2 TXD 3.3V	→	UART2_RX (PA3)
X3 RXD	←	UART2_TX (PA2)
X5 GND	↔	GND
X1	→	GND

3. Bootloader & upload – stap voor stap (UART)

We gebruiken de STM32 system bootloader. Kernregel: BOOT0 moet HIGH zijn tijdens een reset, dan start de bootloader i.p.v. jouw firmware.

3.1 Voorbereiding

- 1 Sluit de TC $\blacksquare$ 9260976 aan zoals in hoofdstuk 2 (TX↔RX gekruist, GND verbonden, X1 op GND).
- 2 Zorg dat de PCB voeding heeft (bij voorkeur via eigen voeding; gebruik niet VBUS van de converter).
- 3 Open Arduino IDE en selecteer het juiste board en COM $\blacksquare$ poort.

3.2 In bootloader mode gaan

- 1 Zet BOOT0 = HIGH (3,3V).
- 2 Reset de MCU: nRST kort naar GND en loslaten.
- 3 Laat BOOT0 HIGH staan zolang je gaat uploaden.

3.3 Uploaden in Arduino IDE

Aanpak (algemeen, kan per IDE/core iets verschillen):

- 1 Tools → Board: STM32 MCU based boards → Generic STM32WL series (of jouw specifieke variant).
- 2 Tools → Port: kies de COM $\blacksquare$ poort van de USB $\blacksquare$ TTL.
- 3 Tools → Upload method: kies UART/STM32CubeProgrammer (UART).
- 4 Klik Upload. Wacht tot 'Done uploading'.

3.4 Na upload: normaal booten

- 1 Zet BOOT0 terug naar GND (LOW).
- 2 Reset opnieuw (nRST kort naar GND).

3 Open Serial Monitor op 115200 om debug te zien.

4. Waarom de LoRaWAN-library eerst niet compileerde (en waarom er patches nodig waren)

Je liep tegen twee typen fouten aan: (A) board-pin definities die niet bij jouw custom RAK3172 passen en (B) een toolchain/sectie-conflict in de LoRaMAC NVM plaatsing.

4.1 RF-switch pins (PC3/PC4/PC5) probleem

De STM32duinoLoRaWAN library bevat BSP code die standaard uitgaat van NUCLEO-WL55 hardware waar RF-switch pinnen op PC3/PC4/PC5 zitten. Op jouw 'Generic STM32WL series' variant bestaan die namen niet (je zag errors als 'PC3 undeclared' en suggesties 'PC_3'). Oplossing voor compile-test: RF-switch pins uitschakelen of aanpassen naar de juiste macro's/pinnen voor jouw board.

4.2 Nvm / NvmBackup section conflict

Je zag errors: 'section of Nvm conflicts with previous declaration'. Dat ontstaat door combinatie van __NO_INIT/ __ROOT met een expliciete section attribute in de GCC-tak. Sommige core+gcc combinaties behandelen dat als dubbele/tegenstrijdige placement. Oplossing: de GCC-variant van Nvm en NvmBackup 'één duidelijke placement' geven (dus óf section attribute, óf NO_INIT placement—maar niet allebei).

4.3 Praktische aanpak die bij jou compilede

Je testte een minimale sketch die wél compilede nadat je een extra header include toevoegde en (tijdelijk) de library-defaults neutraliseerde voor jouw boardvariant. Belangrijk: library-updates kunnen deze wijzigingen overschrijven. Bewaar daarom een patch/diff of kopieer de aangepaste bestanden apart.

5. Debug UART: waarom 'Serial2' soms niet bestaat en hoe we dat robuust oplossen

Je kreeg later linkerfouten 'undefined reference to Serial2'. Dat betekent: jouw boardvariant levert geen global Serial2 object. De oplossing is: maak zelf een HardwareSerial op USART2 en bind PA2/PA3 expliciet. Dit werkt op Generic STM32WL varianten.

5.1 Robuuste debug-constructie

```
// Debug UART on USART2 (PA2/PA3). Works even if Serial2 doesn't exist.
#ifdef USART2
  HardwareSerial DBG(USART2);
#else
  #error "USART2 not available in this board variant"
#endif

void setup() {
  DBG.setTx(PA2);
  DBG.setRx(PA3);
  DBG.begin(115200);
}
```

6. Definitieve testsketch – volledig (met uitleg per blok)

Hieronder staat de volledige testsketch. Deze versie is aangepast zodat hij niet afhankelijk is van Serial2 (dus geen linkerfout), maar wel debug doet op UART2/USART2 met pins PA2/PA3. De logica (payload, sensoren, join, power-kill) is gelijk aan wat we hebben opgebouwd.

6.1 Complete sketch (copy/paste)

```
/*
  RAK3172 (STM32WLE5) definitive test sketch for your PCB
  - Debug over UART2 (PA2/PA3) -> USB-Serial converter
  - BME280 over I2C (SCL=PA12, SDA=PA11)
  - Battery on A0 (PB3) using divider R6=470k, R7=1.5M
  - O2 analog on A1 (PB4)
  - Sends 8-byte binary payload with msgType/version byte (0x01)
*/

#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_BME280.h>
#include <STM32LoRaWAN.h>
#include <math.h>

#include "arduino_secrets.h" // SECRET_APP_EUI / SECRET_APP_KEY

// ===== Pins (per jouw PCB) =====
#define UART2_TX_PIN PA2
#define UART2_RX_PIN PA3

#define I2C_SCL_PIN PA12
#define I2C_SDA_PIN PA11

#define BAT_ADC_PIN A0 // PB3
#define O2_ADC_PIN A1 // PB4

#define PWR_KILL_PIN PA8

// ===== Debug UART (robust) =====
#if defined(USART2)
  HardwareSerial DBG(USART2);
#else
  #error "USART2 is not available in this board variant."
#endif

// ===== LoRaWAN =====
LoRaModem modem;
static const String appEui = SECRET_APP_EUI;
static const String appKey = SECRET_APP_KEY;

// ===== Sensor =====
Adafruit_BME280 bme;

// ===== Battery divider =====
// VBAT -> R6(470k) -> node(A0) -> R7(1.5M) -> GND
constexpr float R6_TOP = 470000.0f;
constexpr float R7_BOT = 1500000.0f;
constexpr float VREF = 3.3f;

// ===== O2 calibration =====
constexpr float O2_VREF = 3.1714f;
constexpr int O2_SAMPLES = 32;

// ===== Timing =====
constexpr uint32_t SEND_INTERVAL_MS = 20UL * 60UL * 1000UL; // 20 min
constexpr bool USE_POWER_KILL_AFTER_SEND = true;

// ===== Payload (8 bytes) =====
#pragma pack(push, 1)
struct PayloadV1 {
  uint8_t msgType; // 0x01
  uint16_t o2_x10; // O2 % * 10
```

```

    int16_t t_x10;        // Temp °C * 10
    uint16_t h_x10;       // RH % * 10
    uint8_t bat_pct;      // 0..100
};
#pragma pack(pop)

static PayloadV1 makePayload(float o2, float t, float h, int batPct) {
    PayloadV1 p{};
    p.msgType = 0x01; // version/msgType byte

    // eenvoudige sanity
    if (!isfinite(o2)) o2 = 0;
    if (!isfinite(t)) t = 0;
    if (!isfinite(h)) h = 0;
    if (o2 < 0) o2 = 0; if (o2 > 25) o2 = 25;
    if (h < 0) h = 0; if (h > 100) h = 100;
    if (t < -100) t = -100; if (t > 150) t = 150;

    p.o2_x10 = (uint16_t)lroundf(o2 * 10.0f);
    p.t_x10 = (int16_t)lroundf(t * 10.0f);
    p.h_x10 = (uint16_t)lroundf(h * 10.0f);
    p.bat_pct = (uint8_t)constrain(batPct, 0, 100);
    return p;
}

static void dumpHex(const uint8_t* b, size_t n) {
    for (size_t i = 0; i < n; i++) {
        if (b[i] < 16) DBG.print('0');
        DBG.print(b[i], HEX);
        DBG.print(' ');
    }
    DBG.println();
}

static float readBatteryVoltage() {
    float vNode = (analogRead(BAT_ADC_PIN) * VREF) / 1023.0f;
    float vBat = vNode * (R6_TOP + R7_BOT) / R7_BOT;
    return vBat;
}

static int batteryPercent(float vBat) {
    long mv = (long)lroundf(vBat * 1000.0f);
    long pct = map(mv, 3200, 4100, 0, 100);
    return (int)constrain(pct, 0, 100);
}

static float readO2Voltage() {
    long sum = 0;
    for (int i = 0; i < O2_SAMPLES; i++) sum += analogRead(O2_ADC_PIN);
    float avg = (float)sum / (float)O2_SAMPLES;
    return avg * (O2_VREF / 1023.0f);
}

static float readO2Percent() {
    float v = readO2Voltage();
    return v * 0.21f / 2.0f * 100.0f;
}

static bool initBME280() {
    Wire.setSDA(I2C_SDA_PIN);
    Wire.setSCL(I2C_SCL_PIN);
    Wire.begin();
    if (bme.begin(0x76)) return true;
    if (bme.begin(0x77)) return true;
    return false;
}

static bool loraJoin() {
    DBG.println("[LoRa] begin(EU868)...");
    if (!modem.begin(EU868)) { DBG.println("[LoRa] modem.begin failed"); return false; }
    DBG.print("[LoRa] version: "); DBG.println(modem.version());
    DBG.print("[LoRa] devEUI : "); DBG.println(modem.deviceEUI());
}

```

```

    DBG.println("[LoRa] joinOTAA...");
    int ok = modem.joinOTAA(appEui, appKey);
    if (!ok) { DBG.println("[LoRa] joinOTAA FAILED"); return false; }
    DBG.println("[LoRa] joinOTAA OK");
    return true;
}

static bool sendPayload(const PayloadV1& p) {
    DBG.print("[LoRa] Sending 8B payload: ");
    dumpHex((const uint8_t*)&p, sizeof(p));

    modem.beginPacket();
    modem.write((const uint8_t*)&p, sizeof(p));
    int err = modem.endPacket(false);

    if (err > 0) { DBG.println("[LoRa] sent OK"); return true; }
    DBG.println("[LoRa] send FAILED");
    return false;
}

static void checkDownlink() {
    if (!modem.available()) return;
    DBG.print("[LoRa] Downlink: ");
    uint8_t buf[64];
    size_t i = 0;
    while (modem.available() && i < sizeof(buf)) buf[i++] = (uint8_t)modem.read();
    dumpHex(buf, i);
}

static void powerKillPulse() {
    pinMode(PWR_KILL_PIN, OUTPUT);
    digitalWrite(PWR_KILL_PIN, LOW);
    delay(50);
    DBG.println("[PWR] PA8 pulse to request power-off...");
    digitalWrite(PWR_KILL_PIN, HIGH);
    delay(200);
    digitalWrite(PWR_KILL_PIN, LOW);
}

void setup() {
    // Debug on UART2
    DBG.setTx(UART2_TX_PIN);
    DBG.setRx(UART2_RX_PIN);
    DBG.begin(115200);
    delay(200);

    DBG.println();
    DBG.println("=== RAK3172 Test Sketch (PCB v4) ===");

    analogReadResolution(10);

    DBG.println("[BME] init...");
    if (!initBME280()) {
        DBG.println("[BME] NOT FOUND (0x76/0x77). Check SDA/SCL wiring and power.");
    } else {
        DBG.println("[BME] OK");
    }

    if (!loraJoin()) {
        DBG.println("[WARN] LoRa join failed. Will retry in 60s...");
    }
}

void loop() {
    static uint32_t lastSend = 0;

    if (!modem.connected()) {
        DBG.println("[LoRa] Not connected -> retry join in 60s");
        delay(60000);
        loraJoin();
        return;
    }
}

```

```

uint32_t now = millis();
if (lastSend == 0 || (now - lastSend) >= SEND_INTERVAL_MS) {
    lastSend = now;

    float t = NAN, h = NAN;
    if (bme.sensorID() != 0) {
        t = bme.readTemperature();
        h = bme.readHumidity();
    }

    float o2 = readO2Percent();
    float vBat = readBatteryVoltage();
    int bat = batteryPercent(vBat);

    DBG.print("[SENS] O2="); DBG.print(o2, 2); DBG.print("% ");
    DBG.print("T="); DBG.print(t, 2); DBG.print("C ");
    DBG.print("RH="); DBG.print(h, 2); DBG.print("% ");
    DBG.print("VBAT="); DBG.print(vBat, 3); DBG.print("V ");
    DBG.print("BAT="); DBG.print(bat); DBG.println("%");

    PayloadV1 p = makePayload(o2, t, h, bat);
    sendPayload(p);
    checkDownlink();

    if (USE_POWER_KILL_AFTER_SEND) {
        powerKillPulse();
        DBG.println("[PWR] Halting (waiting for external power cut / timer)...");
        while (1) { delay(1000); }
    }
}
delay(200);
}

```

6.2 Uitleg – wat doet elk blok? (kort maar stap-voor-stap)

- **Pins:** we leggen PA2/PA3 vast voor UART2 debug, PA11/PA12 voor I2C, A0/A1 voor batterij/O2 en PA8 als power■kill.
- **DBG (USART2):** maakt debug robuust; voorkomt 'undefined reference to Serial2'.
- **PayloadV1 (8B):** vaste layout; byte0 is msgType=0x01 zodat Node■RED kan herkennen welke decoder je moet gebruiken.
- **Sensor reads:** BME280 (temp/RH), O2 via ADC average, battery via dividerformule.
- **LoRa join:** OTAA met appEui/appKey uit arduino_secrets.h.
- **Send:** binary payload met modem.write(pointer, size).
- **Power-kill:** PA8 pulse en daarna halt; alleen gebruiken wanneer hardware timer/power cut getest is.

7. arduino_secrets.h (template)

```

#pragma once
#define SECRET_APP_EUI "0011223344556677"
#define SECRET_APP_KEY "00112233445566778899AABBCCDDEEFF"

```

8. Checklist voor de dag dat het board binnenkomt

- 1 Visuele inspectie: soldeerbruggen, RAK3172 alignment, I2C pullups aanwezig, header pinout klopt.
- 2 Voeding meten: 3V3 aanwezig op MCU, geen onverwachte hoge stroom in idle.
- 3 UART aansluiten: X2→PA3, X3→PA2, X5→GND, X1→GND. Geen X4/X6.
- 4 BOOT test: BOOT0 HIGH + reset → upload werkt; daarna BOOT0 LOW + reset → firmware start.
- 5 Serial monitor 115200: zie boot banner + meetwaarden.
- 6 LoRa join: check devEUI en TTN/ChirpStack instellingen.

7 Eerst zonder power■kill testen; daarna pas USE_POWER_KILL_AFTER_SEND=true.